ie UNIVERSITY | SCHOOL OF SCIENCE & TECHNOLOGY

BACHELOR IN DATA AND BUSINESS ANALYTICS

TIME SERIES ANALYSIS

Session 4 · Decomposition, Transformations and STL

Professor: Angel Mora Segura

24 September 2026 · 18:00–19:30 · Live in-person



COURSE SCHEDULE · 20 SESSIONS · 80 MINUTES EACH

Session by Session

1	02 Sep	Introduction · R setup · forecasting as a business problem ✓
2	03 Sep	Time plots, seasonal plots, EDA ✓
3	23 Sep	Lags, ACF, PACF ✓
4	**24 Sep**	**Decomposition, STL, Box-Cox** **TODAY**
5	30 Sep	Benchmarks: Naive & Seasonal Naive
6	01 Oct	Simple exponential smoothing + Holt
7	07 Oct	Holt-Winters / ETS
8	08 Oct	ETS model selection + validation
9	15 Oct	**Intermediate Test — 20% · Checkpoint 1**
10	19 Oct	AR / MA
11	22 Oct	ARMA + stationarity
12	28 Oct	ARMA + integration + diagnostics
13	29 Oct	ARIMA
14	12 Nov	Seasonal ARIMA (SARIMA)
15	18 Nov	Automatic ARIMA + model selection
16	19 Nov	Complex seasonality / Fourier · Checkpoint 2
17	25 Nov	Integrated Forecasting Workshop
18	26 Nov	Business Case review & feedback
19	03 Dec	**Group presentations — 30%**
20	11 Dec	**Final exam in R — 40%**

TODAY'S SESSION

Taking the Series Apart

Additive and multiplicative schemes, moving averages, variance-stabilising transformations, and STL with its windows.

FPP3 · Chapter 3

TYPICAL CLASS STRUCTURE · 80 MINUTES

The Classroom Is the Laboratory

The same three blocks every session. The work on your case happens here, with me in the room.

~30 MIN

BLOCK 1

Concepts & R application

The methodology, its intuition and its assumptions — aimed at **what problem this technique solves** — then straight into its practical implementation in R.

You leave with code that runs.

~30 MIN

BLOCK 2

Business Case Lab

Teams apply or evaluate the methodology in the context of their own case. Finished code is not the requirement.

Output can be a plot, a methodological decision, a hypothesis, a data transformation, a fitted model, or a justified argument for why this technique does not apply.

~20 MIN

BLOCK 3

Business Case Clinic

Selected teams share their approach, findings or difficulties in about 5 minutes each.

The class discusses alternatives and gives feedback; I close the loop.

Block durations vary with the topic: technically demanding sessions take more time for explanation and R; later sessions shift progressively towards project development and feedback.

BY THE END OF TODAY YOU SHOULD BE ABLE TO

Session Objectives

1. Decide whether your series is **additive or multiplicative**, and justify it from the plot rather than by preference.

2. Use a **moving average** to estimate a trend-cycle, and explain why an even seasonal period needs a centred average.

3. Apply a **log or Box-Cox transformation** to stabilise variance, and choose λ with a defensible argument.

4. Run an **STL decomposition**, tune its trend and season windows, and read the four panels correctly.

5. Produce a **seasonally adjusted series** for your case and say what business question it answers.

BLOCK 1 · 30 MINUTES

Concept + R Application

Three components, two schemes, one transformation family and one algorithm.

Why Take a Series Apart?

A time series is the sum — or the product — of a few simple things happening at once. Estimating them separately is useful long before any model is fitted, because managers ask about the components, not about the series.

01

It answers the question the business actually asks

“Are we growing, or was that just Christmas?” is a question about the trend component. Without decomposition the honest answer is “I cannot tell”.

02

It makes each component inspectable

You can see whether the seasonal shape is stable or drifting, whether the trend has broken, and whether the remainder is white noise or still carries structure.

03

It is a forecasting method in itself

Forecast the seasonally adjusted series, then add the seasonal component back. Simple, transparent, and a genuinely competitive benchmark.

04

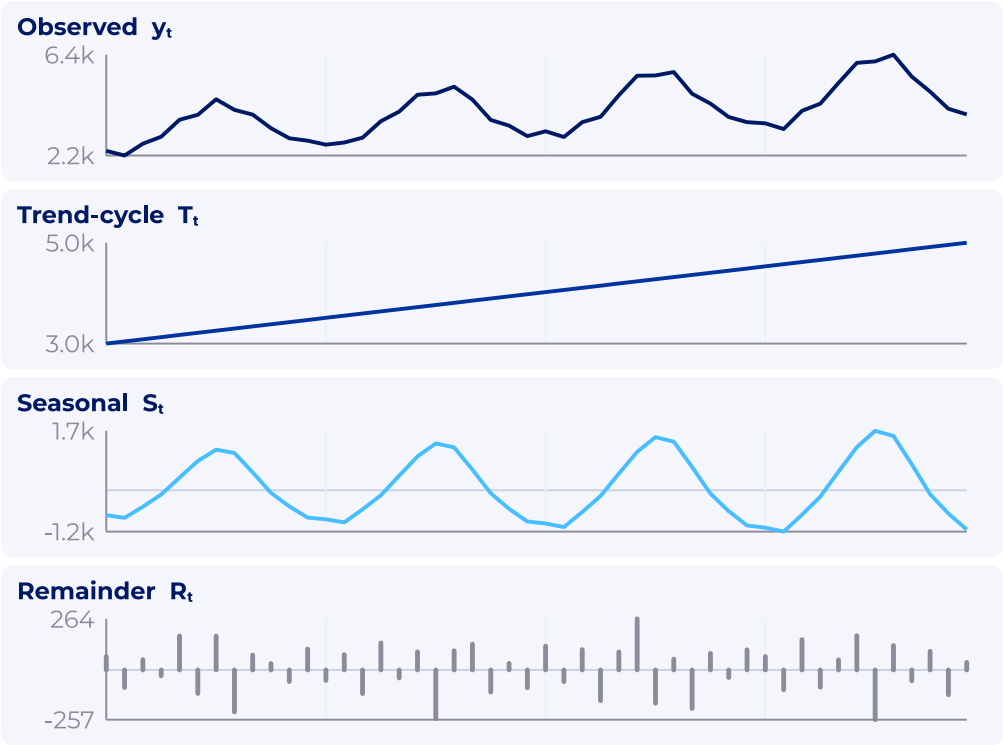
It is how official statistics are published

Unemployment, inflation, retail sales — central banks and statistical offices publish **seasonally adjusted** figures precisely so that month-to-month movements can be interpreted.

Decomposition is diagnostic, communication tool and benchmark at once. That is why it closes the data-understanding block.

EXAMPLE FOUR YEARS OF MONTHLY SALES, PULLED APART

Three Components and a Remainder



Every observation is the three components combined.

- **Trend-cycle.** The long-run movement plus any slow cycle. Deliberately kept together: with a few years of data you cannot separate a rising trend from the upswing of a cycle, so honest practice does not pretend to.
- **Seasonal.** The fixed-period calendar pattern. Here it grows with the level — a clue we will act on shortly.
- **Remainder.** What neither explains. It should look like the white noise of Session 3. If it does not, a component is misspecified.
- Note the vertical scales: the remainder is small, so the two components explain most of the variation. That is the whole judgement in one glance.

Two Schemes, One Decision

You already made this call by eye in Session 2. Here is what it means algebraically, and what it costs to get wrong.

Additive

$$y_t = T_t + S_t + R_t$$

The seasonal effect is a **quantity**: July adds 500 units whether the business sells 3,000 or 30,000. Components are in the same units as the data, so they can be read directly.

Appropriate when the amplitude of the seasonal swing does not change as the level changes — mature, stable operations.

Multiplicative

$$y_t = T_t \times S_t \times R_t$$

The seasonal effect is a **proportion**: July is 1.35 — 35% above trend — at any scale. Seasonal factors are dimensionless and average to 1 over a year.

Appropriate when the swing grows with the level — the usual case in growing businesses and in most economic series.

The bridge between them

- Take logs of a multiplicative series and it becomes additive: `log(T×S×R) = log T + log S + log R`.
- That identity is why transformation comes before decomposition. Transform once, then use additive machinery for everything.
- It is also why a **log scale plot** is worth drawing: on it, a multiplicative series has a constant-width seasonal band.
- Cost of getting it wrong: an additive fit to a growing multiplicative series under-forecasts every future peak, and the error compounds with the horizon.

Rule: if the seasonal band widens as the series rises, you are in the multiplicative world. Transform, or use a multiplicative decomposition — but say which.

Estimating the Trend with a Moving Average

The oldest idea in the subject and still the engine of classical decomposition: replace each observation with the average of its neighbours. Averaging over exactly one full seasonal cycle cancels the seasonality and leaves the trend.



Choosing the order

- **Too short** (3-MA) and the seasonal pattern survives the smoothing — you have removed noise, not seasonality.
- **The order must equal the seasonal period.** A 12-MA on monthly data averages one of every month, so the seasonal effects cancel by construction.
- **Larger order = smoother trend**, but more observations lost at each end and slower response to a genuine turning point.
- **Order matters more than method** here: get the period right and even a crude average gives a usable trend.

A moving average of order equal to the seasonal period is the simplest deseasonalising device that exists — and the base of every classical decomposition.

Why an Even Period Needs Two Passes

With an odd order the average has a natural centre: a 5-MA sits on the third observation. With an even order — 12 months, 4 quarters — the average falls **between** two periods, so it must be re-centred by averaging two consecutive averages.

Step 1 · the 12-MA

$$m_t = (y_{t-6} + y_{t-5} + \ldots + y_{t+5}) / 12$$

Twelve terms, so the centre lands half a month off. The estimate belongs to no month in particular.

Step 2 · centre it

$$T_t = (m_t + m_{t+1}) / 2$$

Averaging two adjacent 12-MAs gives an estimate aligned on month t. The result is written **2×12-MA**, and it is a 13-term weighted average with half weight at each end.

Practical consequences

- **Monthly data → 2×12-MA. Quarterly → 2×4-MA. Daily with a weekly cycle → plain 7-MA**, because 7 is odd and needs no centring.
- **You lose six months at each end.** With 36 observations that is a third of your data gone from the trend estimate — a real cost for Clean Air and Happy Cow.
- **No trend estimate for the most recent months** is the classical method's biggest practical weakness, and precisely where the business needs it. STL and model-based methods do better.

```
# in R you rarely write it by hand
sales |> mutate(
  ma12 = slider::slide_dbl(units, mean,
           .before = 5, .after = 6,
           .complete = TRUE))
```

THE FOUR-STEP RECIPE

Classical Decomposition, Step by Step

1 Estimate the trend-cycle
Compute the 2×m-MA of the series. Call it T_t.

2 Detrend the series
Additive: $y_t - T_t$. Multiplicative: y_t / T_t. What remains is seasonality plus noise.

3 Average by season
For each month, average its detrended values across all years. Twelve numbers, adjusted to sum to zero (or average to one). These are the seasonal indices S_t.

4 The remainder is what is left
$R_t = y_t - T_t - S_t$, or $y_t / (T_t \times S_t)$. Inspect it: it should be structureless.

Worked numbers · July

Detrended July values across four years: **1.34, 1.37, 1.33, 1.36**

Average → $S_{Jul} = 1.35$

July trend next year: **4,900** units

Seasonal forecast for July: 4,900 × 1.35 = **6,615** units

The four values are close together, which is itself the finding: the seasonal pattern is **stable**. If they had read 1.20, 1.28, 1.36, 1.44, the seasonality would be strengthening and a single index would be the wrong summary.

This is the method behind most spreadsheet forecasting in industry. Knowing it lets you audit what a client is already doing — often the first useful thing a consultant delivers.

Why We Do Not Stop There

Classical decomposition is transparent and easy to explain, which is why it survives. But it makes four assumptions that real business data routinely breaks.

No trend at the ends

The first and last m/2 observations get no trend estimate — and the last few months are exactly the ones a planner cares about.

The seasonal pattern is forced to be constant

One index per season, for all years. A business whose summer peak is sharpening every year cannot be represented.

Outliers contaminate everything

A single promotional spike enters the moving average and the seasonal index, distorting both. Nothing is robust.

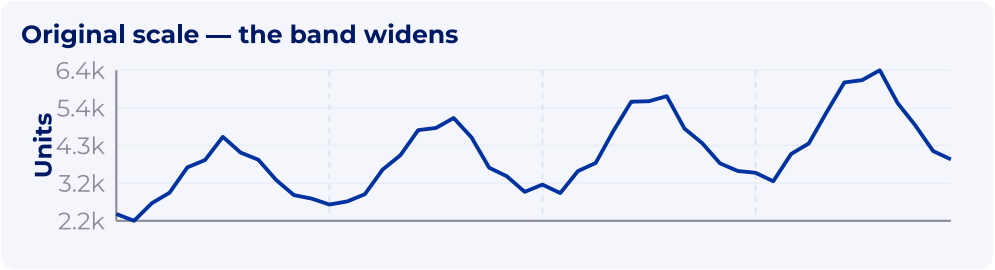
It cannot handle unusual calendars

Moving holidays, trading-day effects, several seasonal periods at once — all outside the method.

STL fixes the first three by construction. That is why it is the default in this course — and why statistical agencies use its industrial cousins, X-13 and SEATS.

Stabilising the Variance: the Logarithm

Before decomposing, make the seasonal band a constant width. The log is the workhorse: it compresses large values more than small ones, so a swing that is proportional on the original scale becomes constant on the log scale.



What the log buys you

- **Multiplicative becomes additive**, so one set of tools handles both worlds.
- **Changes become percentages.** A difference of 0.05 in logs is roughly a 5% change — often exactly how the business talks.
- **Forecast intervals stay positive** when back-transformed, which matters for demand: an additive model on the raw scale can predict negative sales.
- **Two warnings.** Logs are undefined at zero or negative values, so intermittent demand needs another route. And the back-transformed mean is not the mean of the back-transform — R handles the bias adjustment, but know it exists.

Test it visually: plot the series on a log scale. If the seasonal band becomes even, the log is the right transformation and you can stop looking.

The Box-Cox Family: One Parameter, a Whole Spectrum

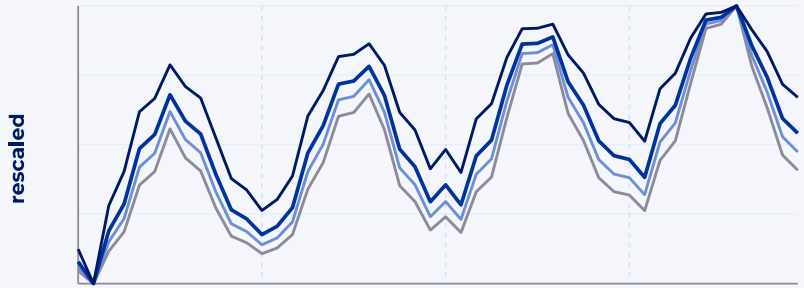
The log is one member of a family indexed by λ. Choosing λ chooses how hard to compress the large values — from no transformation at all to a very strong one.

$$w_t = \log(y_t) \quad \text{if } \lambda = 0$$

$$w_t = (y_t^{\lambda} - 1) / \lambda \quad \text{otherwise}$$

λ = 1	no transformation — the data as it is
λ = 0.5	square root — mild compression
λ = 0	natural log — the common choice
λ = −1	inverse — very strong compression

The same series at four values of λ (rescaled to compare shapes)



As λ falls, the later peaks are pulled down relative to the early ones until the seasonal band is even. Overshoot and you invert the problem: early swings become the wide ones.

Keep λ simple and interpretable. λ = 0 and λ = 0.5 are easy to explain to a client; λ = 0.137 is not, and rarely earns its complexity.

R APPLICATION

Choosing and Applying λ

```
library(fpp3)
# let the data suggest lambda
lam <- sales |>
  features(units, features = guerrero) |>
  pull(lambda_guerrero)
lam
#> 0.12   -> round to 0, use a log
# apply it
sales |> autoplot(box_cox(units, lam))
# or transform inside a model later
# model(ETS(box_cox(units, lam)))
```

The Guerrero method picks the λ that makes the seasonal variation most uniform across years. It is a suggestion, not an instruction.

How to decide, in practice

- **Start with the plot.** If the band is already even, λ = 1: do nothing. A transformation you cannot justify is complexity for free.
- **Round to something explainable.** 0.12 becomes 0, 0.43 becomes 0.5. The difference in accuracy is almost always negligible; the difference in credibility is not.
- **Check both ends of the series.** A λ chosen on a stretch with an outlier will be too aggressive.
- **Remember to back-transform.** Forecasts must be reported in the client's units, with the bias adjustment applied — R does it, but you must say so.
- **Log the decision.** "λ = 0 because the seasonal amplitude grows in proportion to the level; Guerrero suggested 0.12 and we rounded for interpretability."

STL: Seasonal and Trend Decomposition Using Loess

STL estimates the components by repeatedly fitting **local regressions** — loess — rather than by moving averages. That single change removes most of the limitations of the classical method.

What it does better

- **Estimates every observation**, including the first and last, so you get a trend for the current month.
- **Lets the seasonal shape evolve** over the years, at a rate you control — instead of forcing one fixed index per season.
- **Is robust to outliers** when you ask it to be: extreme points can be downweighted so they distort the remainder instead of the trend and season.
- **Handles any seasonal period**, and several at once for high-frequency data.

What it does not do

- **It is additive only.** For multiplicative data, take logs first — which is exactly why the transformation section came before this one.
- **It does not model calendar effects**: trading days, moving holidays like Easter or Chinese New Year. Statistical agencies use X-13ARIMA-SEATS for that.
- **It has no forecasting mechanism of its own.** It decomposes; you then forecast the seasonally adjusted part and add the season back.

```
sales |>
  model(STL(units)) |>
  components() |>
  autoplot()
```

STL is now the standard exploratory decomposition in applied work — it appears in published forecasting of electricity prices, crude oil and epidemic case counts, usually as the first stage of a larger pipeline.

The Two Windows You Actually Tune

STL has two smoothing spans, and they control the same trade-off in two places: how much the component is allowed to change over time. Both are set in **numbers of observations**.

season(window = "periodic") — the shape is frozen



season(window = 7) — the shape is allowed to evolve



How to set them

- **season(window = ...)** — how many years of data inform each seasonal estimate. Small (7–11) lets the pattern drift; `"periodic"` forces it constant, which is the classical assumption.
- **trend(window = ...)** — how smooth the trend is. Larger means smoother and slower to react; smaller follows every wiggle and starts absorbing seasonality.
- **Symptom of a season window too small:** the trend panel goes wobbly, because seasonality is leaking into it.
- **Symptom of a trend window too small:** the remainder shrinks toward zero — the trend is fitting the noise, and you have explained nothing.
- **Diagnose from the remainder.** Adjust the windows until the remainder looks like white noise and neither other panel shows a pattern that belongs to its neighbour.

```
sales |> model(STL(units ~
  trend(window = 21) +
  season(window = 13),
  robust = TRUE))
```

Reading the Four Panels

R draws a grey bar beside each panel showing the same absolute range in every one. The bars are the fastest way to judge which component matters — and students consistently ignore them.

STEP 1

Compare the grey bars

A small bar means a large component: the panel is zoomed in. If the seasonal bar is tiny relative to the trend bar, seasonality dominates the variation.

STEP 2

Look at the trend panel

Is it monotone, or does it turn? A break here is a business event — find it, name it, and decide whether the pre-break data belongs in your estimation sample.

STEP 3

Look at the seasonal panel

Is the shape identical every year, or drifting? A widening shape means the transformation was insufficient. A changing shape means “periodic” is the wrong choice.

STEP 4

Interrogate the remainder

Any visible structure is a failure of the other three. Run its ACF: if bars break the band, a component is misspecified or an event is unmodelled.

One number worth quoting to a client: the share of variance explained by trend versus season. It converts a picture into a sentence about their business.

Seasonally Adjusted Data: What It Is For

Remove the seasonal component and what is left is the trend-cycle plus the remainder. This is the series in which a month-to-month comparison finally means something.

The definition

Additive: $y_t - S_t = T_t + R_t$

Multiplicative: $y_t / S_t = T_t \times R_t$

- **The question it answers:** "Ignoring the fact that January is always weak, is the business actually slowing down?"
- **Who publishes it:** essentially every statistical agency, for unemployment, retail sales, industrial production and inflation.
- **What it is not:** a forecast. It is history, cleaned of a known calendar effect.

The consulting use, per case

Happy Cow. Was the new flavour launch a success, or was it simply July? Only the adjusted series can tell the CEO.

Balearic Airlines. Strip the day-of-week effect and you can see whether underlying demand on the route is genuinely growing.

Clean Air. Separate the COVID collapse in the trend panel from the ordinary seasonal cycle before committing a year of supplier orders.

Wayfair. With thousands of series, decomposition is also a **feature generator**: trend and seasonal strength per item become the inputs that segment the catalogue.

"Sales fell 12% last month" is almost never a finding. "Sales fell 12% after seasonal adjustment" is.

EVERYTHING FROM TODAY, IN ONE PLACE

Session 4 R Cheat Sheet

```
# 1. is a transformation needed?
sales |> autoplot(units)
sales |> autoplot(log(units))
lam <- sales |> features(units,
         features = guerrero) |>
       pull(lambda_guerrero)
# 2. classical decomposition
sales |>
  model(classical_decomposition(units,
        type = "multiplicative")) |>
  components() |> autoplot()
```

```
# 3. STL, tuned
dcmp <- sales |>
  model(STL(box_cox(units, lam) ~
    trend(window = 21) +
    season(window = 13),
    robust = TRUE))
dcmp |> components() |> autoplot()
# 4. seasonally adjusted series
dcmp |> components() |>
  autoplot(season_adjust)
# 5. strength of each component
sales |> features(units, feat_stl)
```

Five Ways Decomposition Goes Wrong

Every one of these has appeared in a real consulting deliverable. Check the list before you show a decomposition to anyone.

Decomposing before transforming	An additive decomposition of a multiplicative series pushes the growing seasonality into the remainder, which then fails every diagnostic. Transform first, always.
Treating the trend panel as a forecast	The trend-cycle is an estimate of the past. Extending it by hand is extrapolation without an error measure — and it is how most spreadsheet forecasts are built.
Ignoring the grey scale bars	Two panels that look equally dramatic can differ by an order of magnitude in size. Without the bars, the plot invites the wrong conclusion.
Fitting a cycle as seasonality	A three-year business cycle in monthly data is not a seasonal pattern. Force it into the seasonal component and it will be projected as if it repeated every year.
Not checking the remainder	It is the free diagnostic. Run its ACF: if structure remains, your decomposition is wrong and every downstream number inherits the error.

BLOCK 2 · 30 MINUTES

Business Case Lab

Transform, decompose, and produce the seasonally adjusted series your client would actually read.

WHAT TO EXPECT IN EACH CASE

Where to Point the Decomposition

CASE 1 · RETAIL

Happy Cow Ice Cream

Decompose daily cups with a weekly period (m = 7). No centring needed — 7 is odd. Expect a strong weekend effect in the seasonal panel.

Watch for: you have under one annual cycle, so the annual seasonality will hide inside the trend panel. Say so explicitly rather than pretending the trend is growth.

CASE 2 · AIRLINES

Balearic Airlines

Decompose daily priority-class demand at m = 7, then look for the National Day week in the remainder panel — that is where an unmodelled holiday lands.

Watch for: set `robust = TRUE`. Without it, the holiday distorts the seasonal indices for every Tuesday of the year.

CASE 3 · MANUFACTURING

Clean Air Limited

Monthly data at m = 12 needs the 2×12-MA, which costs you six months at each end — a fifth of a 36-month series. STL will give you those months back.

Watch for: the trend window is the whole decision here. Too small and COVID looks like the trend; too large and the recovery is smoothed away.

CASE 4 · E-COMMERCE

Wayfair

Do not tune one decomposition — run `features(feat_stl)` across your whole synthetic hierarchy and get trend and seasonal strength per series.

Watch for: those two numbers are a segmentation. Plot them against each other and you have the argument for treating sparse items differently.

BLOCK 3 · 20 MINUTES

Case Clinic

One decomposition per team, and the sentence it produced.

BLOCK 1 COMPLETE · SESSIONS 1 TO 4

Understanding Time Series: Done

SESSION 1

Framed the problem

Forecastability, horizon, granularity, cost of error, and the one-sentence brief.

SESSION 2

Saw the patterns

Time, seasonal, subseries and scatter plots. Additive or multiplicative, decided by eye.

SESSION 3

Measured the structure

Lags, ACF, PACF, white noise and the first warning about stationarity.

SESSION 4

Separated the components

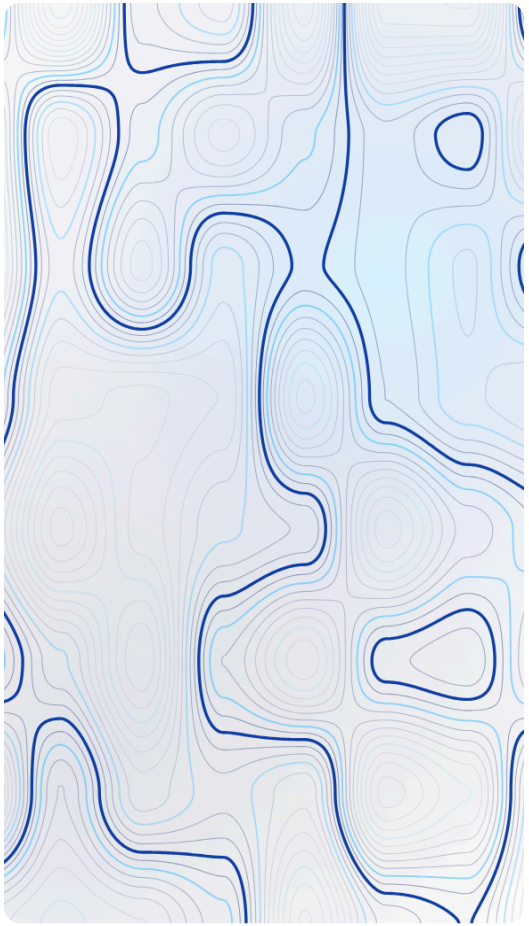
Transformations, moving averages, STL and the seasonally adjusted series.

BLOCK 2 · FROM SESSION 5

Building the baseline

Naive and seasonal naive benchmarks, residual diagnostics, accuracy measures, then the ETS family. Everything from here is judged against a benchmark — including the sophisticated methods.

You now know your data. Nobody in this room can be told “you did not look at it first” — which is the most common criticism of a forecasting project.



END OF SESSION 04

TIME SERIES ANALYSIS

Decomposition, Transformations and STL

NEXT · SESSION 5 · 30 SEPTEMBER

Benchmark methods and forecast accuracy

Block 2 begins. The question changes from “what is in the data?” to “can you beat the benchmark?”

No annex in this session. The R cheat sheet on slide 23 covers everything from today.

Angel Mora Segura · IE University

24 September 2026